

Gliederung

1 Einleitung

In diesem Kapitel wird die Ausgangslage der Baubranche beschrieben, die Problemstellung abgeleitet und das methodische Vorgehen (Design Science Research) definiert.

1.1 Problemstellung

- **Digitalisierungslücke:** Die Baubranche sieht sich im Vergleich zu anderen Industriezweigen mit einem massiven Digitalisierungsrückstand auf der Baustelle konfrontiert.
- **Datensilos und fehleranfällige Prozesse:** Fehlende Interkonnektivität und fehlende Standards zwingen Akteure oft zum fehleranfälligen „Excel-Fallback“.
- **Vendor-Lock und starre Systeme:** Die Bindung an geschlossene Software-Ökosysteme erschwert die Integration spezialisierter Funktionen und blockiert innovative Drittanbieter-Lösungen.
- **Fehlende Netzstabilität:** Eine mangelhafte Mobilfunkabdeckung auf Baustellen führt bei reinen Cloud-Lösungen ohne Offline-First-Strategie zu Datenverlusten und Workflow-Unterbrechungen. (Wird zwar nicht bearbeitet aber der Vollständigkeitshalber)

1.2 Forschungslücke

- **Mangelnde Adaptierbarkeit:** Bestehende Softwarelösungen im BIM-Kontext weisen erhebliche Defizite hinsichtlich ihrer Flexibilität und Anpassbarkeit an spezifische Projektanforderungen auf.
- **Fehlende modulare Architektur:** Es bedarf einer Architektur, die bestehende Datenbestände nahtlos integriert und durch eine einheitliche semantische Datenmodellierung den Informationsfluss gewährleistet.

1.3 Forschungsfragen

- FF1: Wie lässt sich eine Graph-basierte Datenstruktur auf Basis des IFC-Standards gestalten, um die komplexen funktionalen Abhängigkeiten der TGA und MSR

effizient

abzubilden und als Grundlage für ein erweiterbares Inbetriebnahmemanagement zu nutzen?

- FF2: Wie lässt sich eine modulare Multi-Tenant-Architektur für Inbetriebnahmemanagement-Software gestalten, welche funktionale Erweiterungen ermöglicht?

1.4 Zielsetzung und Methodik

- **Primäres Ziel:** Konzeption einer add-on-fähigen Architektur zur Überwindung proprietärer Insellösungen mittels des IFC-Standards und einer performanten Graphdatenbank.
- **Methodisches Vorgehen:** Anwendung des Design-Science-Research-Zyklus von der Anforderungsanalyse über die Entwicklung eines Proof-of-Concepts (PoC) bis hin zur Evaluierung relevanter Metriken.

2 Grundlagen

Dieses Kapitel vermittelt das notwendige technologische Fundament. Es verbindet die BIM-Domäne mit moderner, verteilter Web-Architektur.

2.1 Der IFC-Standard (Building Information Modeling)

- **Zweck und Definition:** Der IFC-Standard (Industry Foundation Classes) ist eine offene Spezifikation zur Beschreibung von Bauelementen und zur Vereinheitlichung der Digitalisierung auf Baustellen[cite: 109, 110].
- **Objekthierarchie:** Darstellung des strukturellen Aufbaus der IFC-Klassen, ausgehend von der Basisklasse IfcRoot bis hin zu spezifischen Bauteilen und Relationen (z. B. IfcRelAggregates)[cite: 112, 113, 127].

2.2 Graphdatenbanken

- **Konzept von Labeled Property Graphs:** Organisation von Daten in Form von Knoten und Kanten, wobei beide Elemente mit Eigenschaften und Labels versehen werden können[cite: 151, 152, 155, 156].
- **Vorteile gegenüber relationalen Datenbanken:** Native und effiziente Darstellung komplexer Beziehungen ohne aufwändige JOIN-Operationen[cite: 153, 173].

- **Neo4j:** Funktionsweise der ACID-konformen Graphdatenbank und Nutzung der deklarativen Abfragesprache Cypher zur gezielten Datenauswertung[cite: 176, 178].

2.3 API-Konzepte: GraphQL vs. REST

- **Verhinderung von Overfetching:** Der Vorteil von GraphQL liegt in der Strukturierung der Anfrage, sodass Clients exakt nur die benötigten Daten erhalten[cite: 198].
- **Kernkonzepte:** Nutzung der stark typisierten Schemadefinitionssprache (SDL) sowie der Einsatz von Resolvern zur Datenabfrage und Mutationen zur Datenveränderung[cite: 204, 205, 206, 213].

2.4 Verteilte Architekturmuster (Backend)

- **CQRS und Event-Sourcing:** Trennung von Schreib- und Lesepfaden im System sowie asynchrone Kommunikation über Domain-Events (Event-Bus) zur Sicherstellung einer losen Kopplung.
- **Multi-Tenant-Datenbankdesign:** Konzepte zur Mandantentrennung (z. B. Row-Level-Security, Tenant-IDs) zur sicheren Isolation von Nutzerdaten bei Großprojekten.

2.5 Moderne Frontend-Technologien

- [cite_start]**Single-Page-Applications mit React:** Nutzung der komponentenorientierten JavaScript-Bibliothek und der Syntaxerweiterung JSX für interaktive Benutzeroberflächen[cite: 216, 217, 218].
- [cite_start]**Microfrontends:** Zerteilung der Benutzeroberfläche in unabhängige Komponenten zur Steigerung von Skalierbarkeit, Flexibilität und Fehlertoleranz[cite: 221, 223].

3. Graphbasiertes Datenbankdesign und API-Middleware

Fundament der Graphdatenstruktur und Erklärung wie reale Baustellendaten strukturiert werden können it GraphQL als "intelligente" Schnittstelle.

3.1 Domänenmodellierung

- **3.1.1 Der Digitale Zwilling:** Definition und Bedeutung des digitalen Zwillings im Inbetriebnahme-Management mittels IFC. Wie wird die physische Baustelle (Gebäude, Räume, Anlagen) als virtueller Knoten-Kanten-Graph abgebildet?
- **3.1.2 Übertragung von IFC-Daten:** Konzepte zur Integration von IFC Files. Wie lassen sich die standardisierten IFC-Strukturen logisch in den Property-Graph (z. B. Neo4j) überführen, um topologische Abhängigkeiten nutzbar zu machen?

3.2 GraphQL als Middleware

- **3.2.1 Typsicherheit:** GraphQL als formaler Vertrag (Schema) zwischen Frontend und Backend. Wie profitiert die Add-On-Architektur davon, dass Datentypen strikt definiert und validiert werden?
- **3.2.2 Revisionssicherheit:** Umsetzung von Historisierung und Auditierbarkeit über die Middleware. (z. B. Wie stellt das API-Gateway sicher, dass Mutationen im Bautagebuch revisionssicher im Graphen protokolliert werden?). GraphQL Mutationen als Sicherheitsnetz um Korruption innerhalb der Graphdatenbank zu vermeiden. Und GraphQL unveränderbar von Dritten um Sicherheit zu gewährleisten
- **3.2.3 Vermeidung von Overfetching:** Der entscheidende Vorteil von GraphQL gegenüber REST unter Baustellenbedingungen (schlechtes Netz). Der Client lädt exakt nur die Daten, die für die aktuelle Ansicht benötigt werden.
- **3.2.4 Flexibler Abfragemöglichkeit** mittels Queries. Vorteile gegenüber starren REST APIs.

3.3 Die allgemeine Schnittstelle

- **3.3.1 Zusammenfassung:** Kurzes Zwischenfazit, das die Vorteile der Graphen-GraphQL-Kombination für die Erweiterbarkeit zusammenfasst und zum Frontend überleitet.

4. Modulare UI-Architektur

Wechseln der Perspektive von Server zu Client und wie die Benutzeroberfläche passend zum flexiblen Backend ebenfalls modular aufgebaut werden kann.

4.1 Einleitung (Microfrontends)

- **Motivation für die UI-Modularisierung:** Warum reicht ein monolithisches React-Frontend für ein add-on-fähiges Inbetriebnahme-System bei Großprojekten nicht aus?

4.2 Arten von Microfrontends

- **Wissenschaftliche Einordnung der verschiedenen Integrationsansätze:** Analyse von Build-Time Integration (z. B. über NPM-Pakete) und Run-Time Integration (z. B. Iframes, Web Components oder serverseitiges Routing).
- **Bewertung der Ansätze:** Evaluierung im Hinblick auf die Anforderungen deiner Arbeit (Unabhängiges Deployment von Add-Ons).

4.3 Umsetzung mit Vite und Module Federation

- **Technologische Entscheidung:** Warum fiel die Wahl auf Module Federation (in Kombination mit Vite)?
- **Funktionsweise:** Implementierung neuer Microfrontends in die App-Shell, Probleme, Deployments und mögliche Automatisierungen
- **Kommunikation zwischen den Frameworks:** Implementierung einer lokalen Kommunikation mittels Local Storage unter der Hilfenahme von JOTAI

5 Verknüpfung der Microfrontends und des Servers

Dieses Kapitel beschreibt die End-to-End-Systemintegration und das konkrete Zusammenspiel zwischen der modularen Benutzeroberfläche (Client) und der Server-Architektur (Backend).

5.1 Kommunikationsmatrix

- **Zentraler Einstiegspunkt:** Rolle der API als Entkopplungsschicht zwischen den Microfrontends im Browser und den Add-On-Modulen.
- **Hybride Kommunikationsprotokolle:** Synthese aus GraphQL (für flexible, overfetching-freie Datenabfragen der UI-Komponenten) und Localstorage für lokale Kommunikation zwischen den Frameworks mittels JOTAI
- **Orchestrierung durch Schema-Stitching:** Technische Umsetzung im Gateway, um die GraphQL-Teilschemata des Core-Systems und die dynamisch registrierten Resolver der Add-Ons zur Laufzeit zu einem einheitlichen API-Endpunkt zusammenzuführen.

6 Zusammenfassung/Fazit

Die Zusammenfassung des Frameworks und ein Fazit der Entwicklung

7 Ausblick

- Ein Ausblick auf mögliche Erweiterungen Support für weitere Frameworks wie Angular Vue etc
- Einbindung einer Middleware neben GraphQL für die Möglichkeit eigener Schnittstellen in REST o.ä.
- Einblick in die Graphentheorie, automatische Erkennung durch Ontologien